

Ex. 1.5 (5 pts)

Why the compiler cannot know. A program is compiled once but run many times, on machines the compiler will never see. Where the binary ends up depends entirely on conditions that exist only at the moment of launch: how much physical memory the machine has, which other processes are already resident, and therefore which free hole the loader happens to pick. Nothing about that is available while the source is being translated. Modern systems push the answer even further away from compile time — address-space layout randomisation deliberately relocates the image on every run, shared libraries are mapped wherever there is room, and demand paging means a page is not given a physical frame until it is first touched. Two runs of the same binary on the same machine can therefore sit at different addresses, so no fixed address could have been correct anyway.

What it assumes instead. The compiler sidesteps the problem by generating code into a *logical* (virtual) address space that it assumes **begins at address 0** and is **contiguous**. Every internal reference — branch targets, function entry points, addresses of globals — is expressed as an offset from that assumed origin, so the whole image can later be shifted anywhere as a unit. The actual binding of logical to physical addresses is deferred to somebody better informed: either a relocating loader that patches the addresses when it places the image, or hardware (a relocation/base register and the MMU) that adds the offset on every reference at run time. This is exactly what makes *relocatable* and *position-independent* code possible. The three binding times are worth naming: *compile time* (absolute code, only usable if the load address is known in advance), *load time* (relocatable code, fixed up once when loaded), and *execution time* (bindings resolved on each reference, which is what lets a process be moved while running — and what essentially all general-purpose operating systems use today).

Ex. 1.6 (5 pts)

The two cases differ *only* in how the command line is assembled; once a path and an argument vector exist, the remaining path through the OS is identical. A double-click makes the desktop environment look up the icon's file association or desktop entry to recover an executable path plus arguments; a typed command makes the shell tokenise the line, expand variables and globs, and resolve the command name against the directories listed in `PATH`. From there:

1. `fork()` (or an equivalent `spawn`) creates a child process, copy-on-write.
2. The child calls `exec()` on the resolved executable path.
3. The kernel checks permissions (execute bit, ownership) and opens the file.
4. It reads the header and validates the **magic number**, which identifies the binary format (ELF, PE, Mach-O) and selects the right loader; a bad magic number fails the `exec` rather than running garbage.

5. The child's old address space is torn down and a fresh one constructed: text, initialised data, zero-filled BSS, heap and stack, with `argv/envp` copied onto the new stack.
6. The segments are set up by memory-mapping the file rather than reading it in, so almost no I/O happens yet.
7. If the binary is dynamically linked, the interpreter named in its header (`ld.so`) is mapped and runs first, mapping the required shared libraries and performing relocations and symbol resolution.
8. The kernel updates the process control block — page tables, inherited file descriptors, reset signal handlers — and puts the process on the ready queue.
9. The scheduler eventually dispatches it and execution starts at the entry point; pages are faulted in on demand as they are actually touched.
10. Runtime startup code initialises the language runtime and then calls `main()`.
11. The program connects to the window system (or inherits the terminal), creates a window and draws into it; the compositor puts those pixels on the screen.

The last difference is on the parent's side: a shell normally `wait()`s for the child (unless it was backgrounded), whereas a GUI launcher usually does not and returns to its event loop immediately.

Ex. 1.7 (5 pts)

Under contiguous allocation each process occupies one unbroken block of physical memory, which makes translation extremely cheap: the entire mapping is described by two registers that the kernel loads on a context switch, the **relocation (base) register** holding the block's starting physical address and the **limit register** holding its length. The MMU applies them in a fixed order — *check first, then add*:

$$\text{physical} = \begin{cases} \text{base} + \text{virtual}, & \text{if } \text{virtual} < \text{limit}, \\ \text{addressing trap to the OS}, & \text{if } \text{virtual} \geq \text{limit}. \end{cases} \quad (1)$$

For example, with `base = 14000` and `limit = 3000`:

$$\begin{aligned} \text{virtual } 346 : 346 < 3000 &\Rightarrow \text{physical} = 14000 + 346 = 14346, \\ \text{virtual } 3200 : 3200 \geq 3000 &\Rightarrow \text{addressing trap (no access)}. \end{aligned}$$

The process only ever names logical addresses in $[0, \text{limit})$ and never sees a physical address, so it is structurally incapable of referring to memory outside its own block — that is where the protection comes from. Loading the base and limit registers is a privileged instruction available only to the kernel, which is what stops a process from simply widening its own bounds.

Ex. 1.8 (5 pts)

Round robin is first-come-first-served made preemptive by a clock. The ready queue is treated as a circular FIFO and each process, on reaching the front, is given the CPU for at most one fixed **time quantum** q (typically tens of milliseconds); if it is still running when the timer interrupt fires, the OS preempts it, saves its context and moves it to the tail of the queue, while a process that blocks or finishes early simply gives the CPU up sooner. With n runnable processes, every one of them therefore receives roughly $1/n$ of the CPU in slices no longer than q and waits no more than about $(n - 1)q$ for its next turn.

The choice of q is the whole design decision: too large and no process is ever actually preempted, so the algorithm degenerates into plain FCFS, while too small and the system spends a growing fraction of its time context-switching instead of doing work — q must stay large relative to the context-switch cost. Its virtues are fairness and a bounded, predictable response time with no possibility of starvation, which is why it suits interactive and time-sharing systems. Its vices are a comparatively poor average turnaround time (worse than SJF) and the fact that it is deliberately blind: it takes no account of priority, deadlines, or how much work a job actually has left.